Creative Software Design
5. Lab1
Compilation & Linking

Dept. of Computer Science

Hanyang University

[IMPORTANT] Submission Guide

- Pull to your project directory before each lab session
  - At your VM instance

```
$ cd Projects/{your directory}
$ git pull origin main
$ cd week05_1
```

- Make sure that your file/directory names are correct
  - Each lab files MUST be placed in ***week05_1***
  - Each lab file names MUST be ***example0x.cpp***
  - You don't have to be creative at this !
- Make sure that your source files are correct
  - No compile errors or warnings
  - Correct output should be appeared in execution (without errors)
  - Any of incorrect files loses all points

Build: Make an Executable File

- **Builds** are typically processed in multiple stages

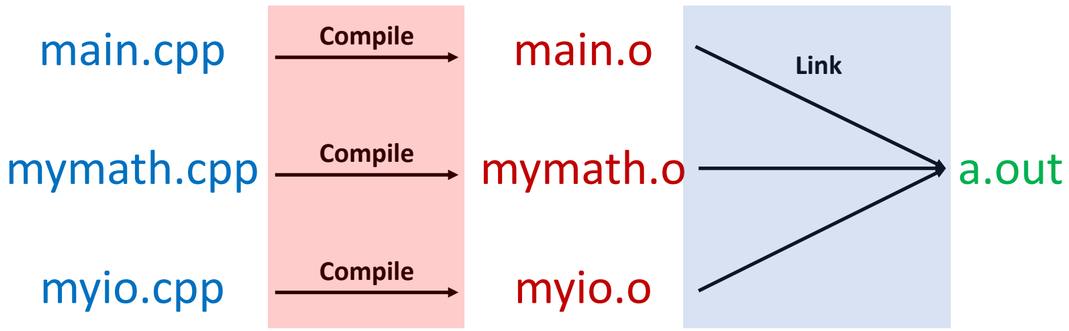
- **Compile**
  - source code → machine code

    ex) main.cpp (source file) → main.o (object file)

- **Linking**
  - Create the final executable file (or library) by linking several object files (+libraries)
    - A library is just a collection of object files.

    ex) main.o, ... → a.out, mylib.so

Compilation & Linking Example



Header & Source Files for Functions

- Function prototypes in a ***header*** file
  - Name, parameters, return type without body
- The actual implementation of functions in a ***source*** file
- Header files provide **interface** for functions
  - Interfaces only need ***how-to-call***, not ***how-they-work***

```
// myfunc.h – header file
int FuncInt(int a, int b);
double Norm(const double* array, int n);
```

```
// myfunc.cpp – source file
#include <math.h>
#include "myfunc.h"

int FuncInt(int a, int b) {
  ...
}
double Norm(const double* array, int n) {
  ...
}
```


#1 – Header & Source Files

- Sepaprate `example01.cpp` into two files

```
// example01.cpp
#include <iostream>

int max(int x, int y) {
  if (x > y) {
    return x;
  } else {
    return y;
  }
}

int main(void) {
  std::cout << max(1, 2) << std::endl;
  return 0;
}
```

```
// max.cpp

int max(int x, int y) {
  if (x > y) {
    return x;
  } else {
    return y;
  }
}
```

```
// example01.cpp
#include <iostream>

int main(void) {
  std::cout << max(1, 2) << std::endl;
  return 0;
}
```

#2 – What Compilers Do

- Compare both compiled and linked binary code

  compile: `$ g++ -c example02.cpp max.cpp`

  linking: `$ g++ example02.o max.o`

- **`objdump`** can help you check assembly code
  - We only check the address of each function call

  `$ objdump –d example02.o | less`

  `$ objdump –d example02 | less`

```
// max.cpp

int max(int x, int y) {
  if (x > y) {
    return x;
  } else {
    return y;
  }
}
```

```
// example02.cpp
#include <iostream>
#include "max.h"

int main(void) {
  std::cout << max(1, 2) << std::endl;
  return 0;
}
```


#3 – What Linkers Do

- Compare both compiled and linked binary code

  `$ g++ -c example03.cpp add.cpp add_dbg.c`

  `$ g++ -o example03.1 lab03.o add.o`

  `$ g++ -o example03.2 lab03.o add_dbg.o`

`lab03.o` was untouched!!

```
// example03.cpp
#include <iostream>
#include "add.h"

int main(void) {
  std::cout << add(1, 2)
    << std::endl;
  return 0;
}
```

```
// add.h
int add(int x, int y);
```

```
// add.cpp
int add(int x, int y) {
  return x + y;
}
```

```
// add_dbg.cpp
#include <iostream>

using namespace std;

int add(int x, int y) {
  cout << "calling add(" << x
    << ", " << y << ")" << endl;
  return x + y;
}
```

#Include Guards

- #include can lead to multiple type declarations
- #include guards prevents re-defining type declarations
  - Standard: **`#ifndef`**
  - Non-standard: **`#pragma once`**

```
// point.h
#ifndef __POINT_H__
#define __POINT_H__

typedef struct
{
    double x;
    double y;
} Point;

#endif
```

```
// point.h
#pragma once

typedef struct
{
    double x;
    double y;
} Point;
```


#4 – Inline Functions

- `inline` does not guarantee that a function will be inlined
- Compare both binary code

```
inline int add(int x, int y) { ... }
__attribute__((always_inline)) inline int add(int x, int y) { ... }
```

```
// example04.cpp
#include <iostream>
#include "addi.h"

using namespace std;

int main(void) {
  int a, b;
  a = add(1, 2);
  b = add(3, 4);
  cout << add(a, b) << endl;
  return 0;
}
```

```
// addi.h
#ifndef __ADDI_H__
#define __ADDI_H__

inline int add(int x, int y) {
  return x + y;
}

#endif
```

#5 - What Preprocessors Do

- Check the output file from the given source code
  - cpp – The C Preprocessor

```
$ cpp test.cpp (or g++ -E test.cpp)
```

```
// example05.cpp
#include <iostream>

#define MAX 10

#define add(x, y) ((x) + (y))

int main(void) {
  int a = 0, b = MAX, result = 0;
  result = add(a, b);
  return 0;
}
```


All Done!!

- Let's submit to ***gitlab***
- Push to ***gitlab*** after each lab session
  - At your VM instance (and make sure **pwd** at "week05_1")

```
# You are at {Your local project directory}/week05_1
$ git add *.cpp *.h
$ git commit –m "submission of week05_1"
$ git push origin main
```